

Muhamad Lingga Darmawan

152024042

IFB-206 KOMPUTASI PARALEL & SISTEM TERDISTRIBUSI BB

dynamic distribution (for uneven NRP), shows when the code expected optimal time

CODE:

```
def dynamic_distribution(tasks, num_workers):
    print(f"=== DYNAMIC DISTRIBUTION ===")
    print(f"Total Tugas: {len(tasks)} | Total Pekerja: {num_workers}\n")

    # Array untuk melacak total waktu pemrosesan yang sedang ditanggung setiap
    # pekerja
    workers_load = [0] * num_workers

    # Simulasi eksekusi tugas satu per satu (dinamis)
    for index, task_time in enumerate(tasks):
        # 1. Mencari pekerja yang memiliki beban paling ringan saat ini
        available_worker = workers_load.index(min(workers_load))

        # 2. Mendistribusikan tugas ke pekerja tersebut
        workers_load[available_worker] += task_time

        # Penjelasan eksekusi setiap tugas (sesuai permintaan soal)
        print(f"[Eksekusi] Tugas ke-{index + 1} (butuh {task_time}s) -> Dialokasikan
        ke Pekerja {available_worker + 1}")
        print(f"        Beban Pekerja {available_worker + 1} sekarang menjadi
        {workers_load[available_worker]}s")

    print("\n=== Hasil Akhir ===")
    for i, load in enumerate(workers_load):
        print(f"Pekerja {i + 1} menyelesaikan seluruh tugas dalam: {load}s")

    # Expected Optimal Time adalah waktu dari pekerja yang selesai paling terakhir
    # (makespan)
    expected_optimal_time = max(workers_load)

    print("-" * 50)
    print(f"[*]      EXPECTED      OPTIMAL      TIME      (Makespan):
    {expected_optimal_time} detik")
    print("-" * 50)

    return expected_optimal_time
```

```

if __name__ == "__main__":
    # Membuat data simulasi: 12 tugas dengan waktu penyelesaian acak 1-10 detik
    simulated_tasks = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 1, 2]

    print(f'Daftar Waktu Tugas yang masuk antrean: {simulated_tasks}\n')

    # Menjalankan fungsi dengan asumsi 3 pekerja/core
    dynamic_distribution(simulated_tasks, num_workers=3)

```

OUTPUT:

```

Daftar Waktu Tugas yang masuk antrean: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

=== DYNAMIC DISTRIBUTION ===
Total Tugas: 10 | Total Pekerja: 3

[Eksekusi] Tugas ke-1 (butuh 1s) -> Dialokasikan ke Pekerja 1
        Beban Pekerja 1 sekarang menjadi 1s
[Eksekusi] Tugas ke-2 (butuh 2s) -> Dialokasikan ke Pekerja 2
        Beban Pekerja 2 sekarang menjadi 2s
[Eksekusi] Tugas ke-3 (butuh 3s) -> Dialokasikan ke Pekerja 3
        Beban Pekerja 3 sekarang menjadi 3s
[Eksekusi] Tugas ke-4 (butuh 4s) -> Dialokasikan ke Pekerja 1
        Beban Pekerja 1 sekarang menjadi 5s
[Eksekusi] Tugas ke-5 (butuh 5s) -> Dialokasikan ke Pekerja 2
        Beban Pekerja 2 sekarang menjadi 7s
[Eksekusi] Tugas ke-6 (butuh 6s) -> Dialokasikan ke Pekerja 3
        Beban Pekerja 3 sekarang menjadi 9s
[Eksekusi] Tugas ke-7 (butuh 7s) -> Dialokasikan ke Pekerja 1
        Beban Pekerja 1 sekarang menjadi 12s
[Eksekusi] Tugas ke-8 (butuh 8s) -> Dialokasikan ke Pekerja 2
        Beban Pekerja 2 sekarang menjadi 15s
[Eksekusi] Tugas ke-9 (butuh 9s) -> Dialokasikan ke Pekerja 3
        Beban Pekerja 3 sekarang menjadi 18s
[Eksekusi] Tugas ke-10 (butuh 10s) -> Dialokasikan ke Pekerja 1
        Beban Pekerja 1 sekarang menjadi 22s

=== Hasil Akhir ===
Pekerja 1 menyelesaikan seluruh tugas dalam: 22s
Pekerja 2 menyelesaikan seluruh tugas dalam: 15s
Pekerja 3 menyelesaikan seluruh tugas dalam: 18s
-----
[*] EXPECTED OPTIMAL TIME (Makespan): 22 detik
-----
PS C:\Users\miling\Documents\kuliah\kompar\load balancing> 

```

- Logika Eksekusi: Seperti yang terlihat pada *capture* kode di atas, sistem tidak membagikan tugas secara bergiliran buta. Setiap kali ada baris [Eksekusi], kode menjalankan perintah `min(workers_load)` untuk mencari pekerja mana yang total beban waktunya paling sedikit pada detik tersebut.
- Alokasi Dinamis: Misalnya, jika Pekerja 1 dan 2 sedang menanggung beban

tugas berat (misal 9 detik dan 10 detik), maka tugas berikutnya secara otomatis akan dialihkan ke Pekerja 3 yang bebannya masih ringan (misal 1 detik).

- Pembaruan Beban (Load Update): Teks Beban Pekerja [X] sekarang menjadi... menunjukkan bahwa sistem langsung mencatat tambahan waktu ke pekerja yang bersangkutan. Data ini digunakan sebagai bahan pertimbangan untuk mendistribusikan tugas antrean selanjutnya.
- Mencapai Optimalitas: Dengan selalu memberikan tugas ke pekerja yang paling "senggang" di setiap iterasi, kode ini berhasil mencegah adanya pekerja yang kelebihan beban (*bottleneck*) sementara pekerja lain menganggur, sehingga Expected Optimal Time (waktu selesai secara keseluruhan) dapat tercapai.